NYC TAXI ANALYTICS · MANUAL DE EJECUCIÓN Y SUSTENTACIÓN

Bases de Datos II

1. CASO DE ESTUDIO

El proyecto analiza registros públicos de viajes de taxi amarillo de la NYC Taxi & Limousine Commission (TLC), de enero a agosto de 2019. Se descargan 8 archivos Parquet, cuyo metadato reporta 57,042,337 viajes, y el catálogo oficial de zonas.

El tamaño descargado no demuestra el requisito de volumen. La verificación se hace en MongoDB con:

db.getSiblingDB("nyc_taxi_analytics").stats(1024 * 1024 * 1024)

Presenta los valores medidos de dataSize, storageSize, indexSize y objects. Solo dataSize cuenta para verificar los 20 GiB; no sumes índices ni uses cifras de ejemplo.
Los informes se limitan a enero-agosto de 2019; las 1,219 fechas de recogida fuera de ese rango se conservan en la colección fuente, pero no afectan las métricas.

2. ARRANQUE EN WINDOWS

Requisitos: MongoDB con Replica Set rs0, Node.js y Python con pyarrow y pymongo.

Descarga:
```
python database\scripts\descargar_dataset_tlc.py
```

Importa todos los viajes:
```
python -m pip install -r database\scripts\requirements.txt
python database\scripts\importar_tlc.py
```

Crea índices tras la importación:
```
mongosh "mongodb://127.0.0.1:27018/nyc_taxi_analytics?replicaSet=rs0" database\indexes\indices_tlc.js
```

Calcula y guarda los informes 1-8 (ejecución inicial; repetir tras reimportar):
```
mongosh "mongodb://127.0.0.1:27018/nyc_taxi_analytics?replicaSet=rs0" database\scripts\materializar_reportes.js
```

Inicia la API desde backend:
```
corepack pnpm install --frozen-lockfile
corepack pnpm start
```

Abre http://localhost:3000.

3. DIEZ INFORMES

1 Viajes e ingresos por día.
2 Zonas con mayor número de recogidas.
3 Rutas origen-destino más transitadas.
4 Ingresos por distrito.
5 Métodos de pago.
6 Distribución de distancias.
7 Demanda por día de semana y hora.
8 Propinas relativas a tarifa por zona.
9 Consultas en lenguaje natural ejecutadas en MongoDB.
10 Viajes recientes con actualización en tiempo real.

El dashboard carga informes al abrir cada pestaña. La API ofrece JSON bajo /api; por ejemplo http://localhost:3000/api/stats.

4. DEMOSTRACIÓN EN TIEMPO REAL

En una terminal aparte:
```
python database\scripts\simulador_tiempo_real.py
```

El script inserta documentos marcados como demo. Change Streams y Socket.io actualizan el

total, el gráfico de zonas y la tabla de viajes recientes. Las filas históricas importadas no se retransmiten como eventos de demo.

5. ÍNDICES Y EXPLAIN

La colección tiene índice para fecha de recogida y viajes recientes; el catálogo tiene índice por distrito y nombre de zona. Ejecuta el análisis con:

```
db.yellow_trips.find({
  pickup_datetime: {
    $gte: ISODate("2019-01-01T00:00:00Z"),
    $lt: ISODate("2019-02-01T00:00:00Z")
  }
}).explain("executionStats")
```

Compara el plan ganador, executionTimeMillis, totalDocsExamined y totalKeysExamined antes y después de crear el índice. Registra resultados de tu propio equipo; tiempos previos de otras máquinas no son evidencia.

6. CONSULTAS NATURALES

Prueba preguntas como:
- ¿Cuáles son las zonas de recogida con más viajes?
- ¿Cuáles son las rutas populares con más viajes?
- ¿Cuáles fueron los ingresos por mes?
- ¿Qué zonas generaron más propinas?

La aplicación muestra la pregunta, el pipeline validado y los resultados consultados en MongoDB. El proveedor puede ser Gemini u Ollama; sin conexión se utilizan consultas guiadas locales.